

## DEPLOYMENT PLAN

**UMHackathon 2026**

[umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

---

# Table of Content

|                                           |           |
|-------------------------------------------|-----------|
| <b>1. Deployment Overview</b>             | <b>3</b>  |
| <b>2. Deployment Architecture</b>         | <b>4</b>  |
| <b>3. Environment Setup</b>               | <b>6</b>  |
| <b>4. CI/CD Pipeline</b>                  | <b>9</b>  |
| <b>5. Deployment Strategy</b>             | <b>11</b> |
| <b>6. Data Management &amp; Migration</b> | <b>12</b> |
| <b>7. Security &amp; Access Control</b>   | <b>14</b> |
| <b>8. Monitoring &amp; Performance</b>    | <b>16</b> |
| <b>9. Rollback &amp; Recovery Plan</b>    | <b>18</b> |
| <b>10. Risk &amp; Mitigation</b>          | <b>20</b> |
| <b>11. Deployment Timeline</b>            | <b>22</b> |

# PeopleGraph — Deployment Documentation

## 1. Deployment Overview

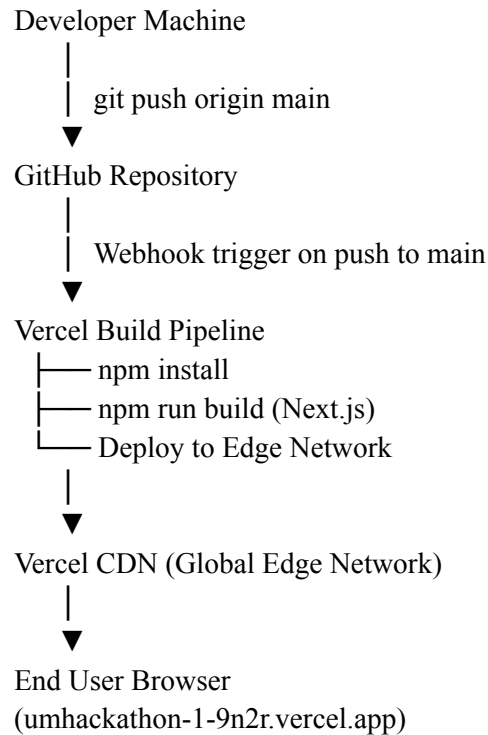
PeopleGraph is a full-stack AI-powered HR intelligence platform built for Malaysian SMEs. The application is deployed as a production web application accessible to end users via a public URL.

| Item                | Details                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------|
| Application Name    | PeopleGraph                                                                                                     |
| Live URL            | <a href="http://umhackathon-1-9n2r.vercel.app">http://umhackathon-1-9n2r.vercel.app</a>                         |
| Deployment Platform | Vercel                                                                                                          |
| Repository          | <a href="https://github.com/TeohXiXian-2025/umhackathon-1">https://github.com/TeohXiXian-2025/umhackathon-1</a> |
| Primary Branch      | main                                                                                                            |
| Framework           | Next.js 16 (Frontend), FastAPI (Backend)                                                                        |
| Database            | PostgreSQL 16                                                                                                   |
| Deployment Type     | Continuous Deployment (auto-deploy on push)                                                                     |
| Environment         | Production                                                                                                      |

The frontend is deployed on Vercel's global CDN edge network, ensuring fast load times worldwide. The deployment pipeline is fully automated — any code pushed to the **main** branch triggers an immediate build and deployment with zero downtime.

## 2. Deployment Architecture

### System Architecture Diagram



### Component Breakdown

| Layer         | Technology            | Platform              | Purpose                            |
|---------------|-----------------------|-----------------------|------------------------------------|
| Frontend      | Next.js 16, React 19  | Vercel                | UI, routing, server-side rendering |
| Styling       | Tailwind CSS 4        | Bundled with frontend | Responsive UI design               |
| Visualization | D3.js                 | Bundled with frontend | Force-directed value graph         |
| Backend       | FastAPI (Python 3.12) | Docker / VPS          | REST API, business logic           |
| Database      | PostgreSQL 16         | Docker volume         | Persistent data storage            |
| Reverse Proxy | Nginx 1.27            | Docker                | Route traffic to services          |

## Directory Structure

```
umhackathon-1/
├── frontend/      ← Deployed to Vercel
│   ├── app/      ← Next.js App Router pages
│   │   ├── components/ ← React components
│   │   ├── lib/    ← Utility functions
│   │   ├── public/ ← Static assets
│   │   └── package.json
│   └── backend/   ← FastAPI application
│       ├── app/
│       │   ├── routers/ ← API endpoints
│       │   ├── models/  ← Database models
│       │   ├── services/ ← Business logic
│       │   └── schemas/ ← Data validation
│       └── Dockerfile
├── deploy/
│   └── nginx/
│       └── default.conf
├── docker-compose.yml    ← Local development
└── docker-compose.prod.yml ← Production
```

### 3. Environment Setup

#### Frontend Environment Variables

Set in Vercel Dashboard → Project → Settings → Environment Variables:

| Variable            | Required | Description              | Example                                                             |
|---------------------|----------|--------------------------|---------------------------------------------------------------------|
| NEXT_PUBLIC_API_URL | Yes      | Backend API base URL     | <a href="https://api.yourdomain.com">https://api.yourdomain.com</a> |
| GOOGLE_API_KEY      | Optional | Google AI Studio API key | <a href="#">AIza...</a>                                             |

Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser at build time. All others are server-side only and never exposed to clients.

## Backend Environment Variables

Stored in `backend/.env.prod` (never committed to Git):

| Variable          | Required | Description                           | Example                                   |
|-------------------|----------|---------------------------------------|-------------------------------------------|
| APP_NAME          | Yes      | Application name                      | PeopleGraph                               |
| DEBUG             | Yes      | Debug mode                            | false                                     |
| SECRET_KEY        | Yes      | JWT signing secret (min 32 chars)     | openssl rand -hex 32                      |
| DATABASE_URL      | Yes      | Async PostgreSQL connection string    | postgresql+asyncpg://user:pass@host/db    |
| POSTGRES_USER     | Yes      | Database username                     | peoplegraph                               |
| POSTGRES_PASSWORD | Yes      | Database password                     | <strong-password>                         |
| POSTGRES_DB       | Yes      | Database name                         | peoplegraph                               |
| CORS_ORIGINS      | Yes      | Allowed frontend origins (JSON array) | ["https://umhackathon-1-9n2r.vercel.app"] |
| AI_AGENT_URL      | Optional | AI agent endpoint                     | https://api.zai.ytl.com/v1/...            |
| AI_AGENT_KEY      | Optional | AI Agent API key                      | <key>                                     |

### Malaysian Statutory Rate Variables (Backend)

| Variable                          | Default | Description                                 |
|-----------------------------------|---------|---------------------------------------------|
| EPF_EMPLOYEE_RATE_BELOW_60        | 0.11    | EPF employee contribution (11%)             |
| EPF_EMPLOYER_RATE_SALARY_LTE_5000 | 0.13    | EPF employer rate for salary $\leq$ RM5,000 |
| EPF_EMPLOYER_RATE_SALARY_GT_5000  | 0.12    | EPF employer rate for salary $>$ RM5,000    |
| SOCISO_EMPLOYER_RATE              | 0.0175  | SOCISO employer contribution                |
| SOCISO_EMPLOYEE_RATE              | 0.005   | SOCISO employee contribution                |
| EIS_RATE                          | 0.002   | EIS rate (both employer and employee)       |

### Setting Up Vercel Environment Variables

1. Log in to <https://vercel.com/dashboard>
2. Select the **umhackathon-1** project
3. Go to **Settings** → **Environment Variables**
4. Click **Add New** for each variable
5. Set **Environment** to **Production**, **Preview**, and **Development** as needed
6. Click **Save**
7. Trigger a redeployment for changes to take effect

## 4. CI/CD Pipeline

### Pipeline Overview

PeopleGraph uses a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline powered by **GitHub** and **Vercel**.

Code Change → GitHub Push → Vercel Webhook → Build → Test → Deploy → Live

### Trigger Conditions

| Event                              | Action                          |
|------------------------------------|---------------------------------|
| Push to <b>main</b> branch         | Full production deployment      |
| Push to feature branch             | Preview deployment (unique URL) |
| Pull Request opened                | Preview deployment for review   |
| Pull Request merged to <b>main</b> | Production deployment           |

### Build Pipeline Steps

| Step                   | Tool            | Description                                 |
|------------------------|-----------------|---------------------------------------------|
| 1. Code checkout       | Vercel / GitHub | Clones the latest commit                    |
| 2. Dependency install  | npm             | Runs <b>npm install</b> in <b>frontend/</b> |
| 3. Build               | Next.js         | Runs <b>npm run build</b>                   |
| 4. Static optimization | Next.js         | Optimizes images, fonts, and assets         |
| 5. Deploy              | Vercel          | Pushes build to edge network globally       |
| 6. Assignment          | Vercel          | Points production domain to new deployment  |

## Build Configuration (Vercel)

| Setting          | Value         |
|------------------|---------------|
| Root Directory   | frontend/     |
| Build Command    | npm run build |
| Output Directory | .next         |
| Install Command  | npm install   |
| Node.js Version  | 20.x          |

## Average Pipeline Duration

| Phase                | Duration            |
|----------------------|---------------------|
| Install dependencies | ~30–60 seconds      |
| Build (Next.js)      | ~60–120 seconds     |
| Deploy to edge       | ~30–60 seconds      |
| <b>Total</b>         | <b>~2–4 minutes</b> |

## Preview Deployments

Every branch and pull request gets its own preview URL automatically:

- Format: `umhackathon-1-<hash>.vercel.app`
- Allows testing changes before merging to `main`
- Deleted automatically after the branch is merged or closed

## 5. Deployment Strategy

### Strategy: Blue-Green via Vercel Atomic Deployments

Vercel uses **atomic deployments** — the new build is fully built and ready before traffic is switched over. This means:

- Zero downtime during deployments
- Instant rollback if an issue is detected
- No partial states visible to users

### Deployment Flow

Step 1: Developer pushes to main



Step 2: Vercel builds the new version in isolation



Step 3: New version passes build checks



Step 4: Traffic is atomically switched to new version



Step 5: Old version is retained for rollback (30 days)

### Branching Strategy

| Branch    | Purpose                     | Auto-Deploy                |
|-----------|-----------------------------|----------------------------|
| main      | Production-ready code       | Yes → Production           |
| feature/* | New features in development | Yes → Preview URL          |
| fix/*     | Bug fixes                   | Yes → Preview URL          |
| hotfix/*  | Urgent production fixes     | Merge to main → Production |

### Deployment Checklist (Before Each Push)

- ☐ Changes tested locally (npm run dev)
- ☐ No console errors in browser
- ☐ All features working as expected
- ☐ Environment variables updated if needed
- ☐ Commit message is descriptive
- ☐ Pushed to correct branch

## 6. Data Management & Migration

© UMHackathon 2026

Email: [umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

## Current Data Architecture

| Layer            | Technology              | Storage                           |
|------------------|-------------------------|-----------------------------------|
| Primary Database | PostgreSQL 16           | Docker persistent volume (pgdata) |
| Development DB   | SQLite (peoplegraph.db) | Local file in backend/            |
| ORM              | SQLAlchemy (async)      | Python                            |
| Migrations       | Alembic                 | backend/alembic/                  |

## Database Schema Overview

The database contains the following core tables:

| Table            | Description                                       |
|------------------|---------------------------------------------------|
| departments      | Company departments (multilingual: EN, BM, ZH)    |
| employees        | Employee records with salary, role, work location |
| users            | System users with role-based access               |
| roles            | Permission roles (owner, hr_admin, etc.)          |
| attendance_logs  | Daily attendance records with hours worked        |
| sentiment_scores | Weekly team sentiment scores by department        |

## Running Migrations

```
# Apply all pending migrations
alembic upgrade head
# Create a new migration after model changes
alembic revision --autogenerate -m "description of change"
# Rollback one migration
alembic downgrade -1
```

## Seeding Demo Data

The application ships with a built-in seed endpoint that populates the database with 10 sample Malaysian employees across 5 departments, 30 days of attendance logs, and 4 weeks of sentiment scores.

```
# Seed via HTTP (after backend is running)
```

© UMHackathon 2026

Email: [umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

```
curl -X POST http://localhost:8000/api/v1/seed
```

```
# Or via PowerShell
```

```
Invoke-RestMethod -Method POST -Uri "http://localhost:8000/api/v1/seed"
```

## Backup Strategy

| Frequency              | Method                   | Retention             |
|------------------------|--------------------------|-----------------------|
| Daily                  | pg_dump automated backup | 7 days                |
| Before each deployment | Manual snapshot          | Until next deployment |
| Weekly                 | Full database export     | 30 days               |

### Manual backup command:

```
docker exec <db-container> pg_dump -U peoplegraph peoplegraph > backup_$(date +%Y%m%d).sql
```

## Restore from Backup

```
docker exec -i <db-container> psql -U peoplegraph peoplegraph < backup_20260502.sql
```

## 7. Security & Access Control

### Frontend Security (Vercel)

| Measure                | Implementation                                               |
|------------------------|--------------------------------------------------------------|
| HTTPS                  | Enforced by default on all Vercel deployments                |
| Environment secrets    | Stored encrypted in Vercel, never exposed in code            |
| NEXT_PUBLIC_ variables | Only non-sensitive values are prefixed (client-exposed)      |
| Headers                | Vercel auto-injects security headers (X-Frame-Options, etc.) |

### Backend Security

| Measure            | Implementation                                                                        |
|--------------------|---------------------------------------------------------------------------------------|
| JWT Authentication | HS256 signed tokens, configurable expiry                                              |
| CORS               | Strict allow-list via <code>CORS_ORIGINS</code> environment variable                  |
| Secret management  | All secrets in <code>.env.prod</code> , excluded from Git via <code>.gitignore</code> |
| Input validation   | Pydantic schemas enforce strict request/response typing                               |
| SQL injection      | SQLAlchemy ORM — no raw SQL queries                                                   |

### CORS Configuration

The backend is configured to only accept requests from explicitly listed origins:

```
CORS_ORIGINS=["https://umhackathon-1-9n2r.vercel.app"]
```

Any request from an unlisted origin will be rejected with a `403 Forbidden` response.

## Access Control Roles

| Role     | Permissions                                                               |
|----------|---------------------------------------------------------------------------|
| owner    | Full access: financials, payroll, analytics, employees, compliance export |
| hr_admin | Payroll, employee management, analytics, compliance export                |

## Secrets That Must Never Be Committed to Git

- SECRET\_KEY
- POSTGRES\_PASSWORD
- ILMU\_API\_KEY
- GOOGLE\_API\_KEY
- Any .env.prod or .env files

All secrets are listed in .gitignore to prevent accidental exposure.

## 8. Monitoring & Performance

### Vercel Built-in Monitoring

Vercel provides the following observability tools out of the box:

| Feature               | Location                   | Description                           |
|-----------------------|----------------------------|---------------------------------------|
| <b>Build Logs</b>     | Dashboard → Deployments    | Full build output for each deployment |
| <b>Runtime Logs</b>   | Dashboard → Logs           | Real-time server-side logs            |
| <b>Analytics</b>      | Dashboard → Analytics      | Page views, visitors, Web Vitals      |
| <b>Speed Insights</b> | Dashboard → Speed Insights | Core Web Vitals per page              |

### Key Health Endpoints

| Endpoint                  | Method | Expected Response                         |
|---------------------------|--------|-------------------------------------------|
| /health                   | GET    | { "status": "healthy", "version": "..." } |
| /api/v1/dashboard/summary | GET    | Dashboard KPI data                        |
| /docs                     | GET    | Swagger API documentation UI              |

### Performance Targets

| Metric                         | Target      | Description                          |
|--------------------------------|-------------|--------------------------------------|
| First Contentful Paint (FCP)   | < 1.8s      | Time until first content appears     |
| Largest Contentful Paint (LCP) | < 2.5s      | Time until main content loads        |
| Time to Interactive (TTI)      | < 3.5s      | Time until page is fully interactive |
| API Response Time              | < 500ms     | For standard dashboard endpoints     |
| Build Time                     | < 5 minutes | Full CI/CD pipeline duration         |

### Monitoring Checklist (Post-Deployment)

- [ ] Homepage loads without errors
- [ ] Build completed successfully in Vercel dashboard
- [ ] No runtime errors in Vercel Logs
- [ ] /health endpoint returns { "status": "healthy" }

© UMHackathon 2026

Email: [umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

- [ ] D3 Force Graph renders correctly
- [ ] All navigation links work
- [ ] Browser console shows no errors (F12)
- [ ] Page load time is acceptable on mobile

## 9. Rollback & Recovery Plan

### Rollback via Vercel Dashboard (Recommended)

Vercel retains all previous deployments for 30 days. Rolling back is instant.

#### Steps:

1. Go to <https://vercel.com/dashboard>
2. Select the **umhackathon-1** project
3. Click the **Deployments** tab
4. Find the last known-good deployment
5. Click ... (three dots) next to that deployment
6. Select **Promote to Production**
7. The live URL immediately reverts to that build

**Time to recover:** < 1 minute

### Rollback via Git

# Find the commit to roll back to  
`git log --oneline -10`

# Revert the last commit (creates a new commit, safe for shared repos)  
`git revert HEAD`  
`git push origin main`

# Or hard reset to a specific commit (use with caution)  
`git reset --hard <commit-hash>`  
`git push --force origin main`

### Database Recovery

| Scenario              | Action                                          |
|-----------------------|-------------------------------------------------|
| Bad migration applied | Run <code>alembic downgrade -1</code> to revert |
| Data corruption       | Restore from latest <code>pg_dump</code> backup |
| Accidental deletion   | Restore from point-in-time backup               |

#### Restore from backup:

```
docker exec -i <db-container> psql -U peoplegraph peoplegraph < backup_YYYYMMDD.sql
```

© UMHackathon 2026

Email: [umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)

**Recovery Priority Order**

| Priority | Scenario                    | Recovery Time Target                    |
|----------|-----------------------------|-----------------------------------------|
| P1       | Frontend not loading        | < 5 minutes (Vercel rollback)           |
| P2       | API returning errors        | < 15 minutes (backend restart/rollback) |
| P3       | Database connection failure | < 30 minutes (restart/restore)          |
| P4       | Data corruption             | < 2 hours (backup restore)              |

## 10. Risk & Mitigation

| #  | Risk                                 | Likelihood | Impact   | Mitigation                                                                            |
|----|--------------------------------------|------------|----------|---------------------------------------------------------------------------------------|
| 1  | Build failure on push                | Medium     | Medium   | Review build logs in Vercel; fix errors locally before pushing                        |
| 2  | Environment variable missing         | Low        | High     | Maintain <code>.env.example</code> files; verify variables in Vercel before deploy    |
| 3  | Database connection failure          | Low        | High     | Use health checks in Docker Compose; configure retry logic                            |
| 4  | API rate limit exceeded (YTL/Google) | Medium     | Medium   | Implement request caching; handle errors gracefully in UI                             |
| 5  | CORS misconfiguration                | Low        | High     | Always update <code>CORS_ORIGINS</code> when frontend URL changes                     |
| 6  | Secret key exposed in code           | Very Low   | Critical | <code>.gitignore</code> enforced; pre-commit hooks recommended                        |
| 7  | Dependency vulnerability             | Medium     | Medium   | Regularly run <code>npm audit</code> and <code>pip audit</code> ; update dependencies |
| 8  | Vercel free tier limits exceeded     | Low        | Medium   | Monitor usage in Vercel dashboard; upgrade plan if needed                             |
| 9  | Data loss from bad migration         | Low        | Critical | Always backup database before running migrations                                      |
| 10 | Next.js build breaking changes       | Low        | Medium   | Pin dependency versions; test on preview URL before merging                           |

## Risk Response Procedures

### If the site goes down:

1. Check Vercel Status page: <https://vercel-status.com>
2. Check Vercel build/runtime logs for errors
3. If a bad deployment: rollback immediately via Vercel dashboard
4. Notify the team via group chat with status update

### If API is unreachable:

1. Check backend container status: `docker compose ps`
2. Check backend logs: `docker compose logs -f backend`
3. Restart backend: `docker compose restart backend`
4. If issue persists: redeploy from last stable Docker image

## 11. Deployment Timeline

### Initial Deployment History

| Date       | Version | Description                         | Status |
|------------|---------|-------------------------------------|--------|
| April 2026 | v1.0    | Initial full-stack deployment       | ✓ Live |
| April 2026 | v1.1    | Strategic Insight Panel added       | ✓ Live |
| April 2026 | v1.2    | Cuti Peristiwa Shock Simulation     | ✓ Live |
| May 2026   | v1.3    | README and deployment documentation | ✓ Live |

### Deployment Process Timeline (Each Release)

| Time   | Phase       | Action                                       |
|--------|-------------|----------------------------------------------|
| T+0:00 | Code ready  | Developer pushes to <span>main</span> branch |
| T+0:01 | Trigger     | Vercel detects push via GitHub webhook       |
| T+0:01 | Build start | Vercel clones repo, installs dependencies    |
| T+0:02 | Compile     | Next.js compiles pages and components        |
| T+0:03 | Optimize    | Static assets optimized, images processed    |
| T+0:04 | Deploy      | Build pushed to Vercel global edge network   |
| T+0:04 | Go live     | Production URL updated to new build          |
| T+0:05 | Verify      | Team verifies live URL is working correctly  |

## Future Deployment Milestones

| Phase                           | Action                                                                                                                    | Timeline   |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Phase 1 — Infrastructure</b> | Cloud server provisioning, pgvector setup, Docker registry initialisation, environment variable configuration             | Week 1–2   |
| <b>Phase 2 — Testing</b>        | Staging environment deployment, compliance logic unit tests, load testing, RBAC security audit, AI integration validation | Week 3–4   |
| <b>Phase 3 — Soft Launch</b>    | Pilot onboarding for <b>50 Klang Valley SMEs</b> — payroll and compliance modules live                                    | Month 1–4  |
| <b>Phase 4 — Expansion</b>      | Penang and Johor rollout — anomaly detection, sentiment analysis, and dashboard modules activated                         | Month 4–12 |

---

*Document maintained by the PeopleGraph development team.*

Repository: <https://github.com/TeohXiXian-2025/umhackathon-1>

Last updated: 02 May 2026